



南開大學
Nankai University

数据结构实验报告

(2024~2025 学年第二学期)

实验名称：表达式求值与路径简化

学 院：软件学院

姓 名：郭威威

学 号：2414308

指导老师：刘明铭

2024~2025 春季学期

使用 L^AT_EX 撰写于 2025 年 3 月 12 日

目录

1	表达式求值实验	2
1.1	算法原理	2
1.1.1	中缀转后缀算法	2
1.1.2	后缀表达式计算	2
1.2	测试结果	2
2	路径简化实验	3
2.1	算法实现	3
2.1.1	核心逻辑	3
2.2	测试案例	3
3	实验分析	4
3.1	关键数据结构	4
3.1.1	栈模板类实现	4
3.2	复杂度对比	5
A	完整源代码	6
A.1	表达式求值程序	6
A.2	路径简化程序	10

实验一 表达式求值实验

1.1 算法原理

1.1.1 中缀转后缀算法

- 运算符优先级处理：使用栈临时存储运算符
- 数字处理：连续数字和小数点识别
- 错误检测：括号匹配检查和非法字符识别

1.1.2 后缀表达式计算

- 操作数栈管理：支持浮点数运算
- 异常处理：除零错误检测
- 运算支持：六种运算符 (+, -, *, /, %, ^)

1.2 测试结果

测试用例	预期输出	实际输出
$2.5 + 2^3 / 2 \% 3$	3.50	3.50
$9 + (3 - 1) * 3 + 10 / 2$	20.00	20.00
$5.2 * 5.8 / 3) + 2$	ERROR	ERROR

表 1.1: 表达式求值测试结果

实验二 路径简化实验

2.1 算法实现

2.1.1 核心逻辑

- 使用 vector 模拟栈结构
- 路径分割：按 '/' 分割路径组件
- 特殊符号处理：识别 "." 和 ".."

2.2 测试案例

```
1 输入路径： /a//b////c/d//././../..
2 输出结果： /a/b/c
3 处理过程：
4 1. 分割组件： [ "a", "b", "c", "d", ".", ".", ".." ]
5 2. 栈操作： a->b->c (遇到..弹出d)
```

实验三 实验分析

3.1 关键数据结构

3.1.1 栈模板类实现

```
1  template<typename T>
2  class Stack {
3  private:
4      T* data;          // 存储元素数组
5      int top;          // 栈顶指针
6      int capacity;     // 栈容量
7
8  public:
9      Stack(int size=10) {
10         capacity = size;
11         data = new T[capacity];
12         top = -1;
13     }
14
15     void push(T val) {
16         if(top >= capacity-1) resize();
17         data[++top] = val;
18     }
19
20     T pop() {
21         if(isEmpty()) throw "Stack Underflow";
22         return data[top--];
23     }
```

24

};

3.2 复杂度对比

算法	时间复杂度	空间复杂度	稳定性
表达式求值	$O(n)$	$O(n)$	稳定
路径简化	$O(n)$	$O(n)$	稳定

实验一 完整源代码

A.1 表达式求值程序

```
1 #include <iostream>
2 #include <stack>
3 #include <string>
4 #include <sstream>
5 #include <cmath>
6 #include <iomanip>
7 using namespace std;
8
9 // 判断字符是否为操作符
10 bool isOperator(char c) {
11     return c == '+' || c == '-' || c == '*' || c == '/' || c == '%' ...
12         || c == '^';
13 }
14
15 // 获取操作符优先级
16 int getPriority(char op) {
17     switch (op) {
18         case '^': return 3;
19         case '*': case '/': case '%': return 2;
20         case '+': case '-': return 1;
21         default: return 0;
22     }
23 }
24
25 // 中缀表达式转后缀表达式
26 string infixToPostfix(const string& infix) {
27     stack<char> operatorStack;
28     string postfix = "";
```

```
29     for (size_t i = 0; i < infix.size(); ++i) {
30         char c = infix[i];
31
32         if (isdigit(c) || c == '.') {
33             // 处理操作数
34             while (i < infix.size() && (isdigit(infix[i]) || ...
35                 infix[i] == '.')) {
36                 postfix += infix[i];
37                 ++i;
38             }
39             postfix += ' ';
40             --i;
41         }
42         else if (c == '(') {
43             operatorStack.push(c);
44         }
45         else if (c == ')') {
46             while (!operatorStack.empty() && operatorStack.top() != ...
47                 '(') {
48                 postfix += operatorStack.top();
49                 postfix += ' ';
50                 operatorStack.pop();
51             }
52             if (operatorStack.empty()) {
53                 return ""; // 括号不匹配
54             }
55             operatorStack.pop(); // 弹出 '('
56         }
57         else if (isOperator(c)) {
58             while (!operatorStack.empty() && ...
59                 getPriority(operatorStack.top()) >= getPriority(c)) {
60                 postfix += operatorStack.top();
61                 postfix += ' ';
62                 operatorStack.pop();
63             }
64             operatorStack.push(c);
65         }
66         else if (isspace(c)) {
```



```
64         continue;
65     }
66     else {
67         return ""; // 非法字符
68     }
69 }
70
71 while (!operatorStack.empty()) {
72     if (operatorStack.top() == '(') {
73         return ""; // 括号不匹配
74     }
75     postfix += operatorStack.top();
76     postfix += ' ';
77     operatorStack.pop();
78 }
79
80 return postfix;
81 }
82
83 // 计算后缀表达式的值
84 double evaluatePostfix(const string& postfix) {
85     stack<double> operandStack;
86     istringstream iss(postfix);
87     string token;
88
89     while (iss >> token) {
90         if (isdigit(token[0]) || token[0] == '.') {
91             double num;
92             istringstream(token) >> num;
93             operandStack.push(num);
94         }
95         else if (isOperator(token[0])) {
96             if (operandStack.size() < 2) {
97                 return NAN; // 操作数不足
98             }
99             double operand2 = operandStack.top();
100             operandStack.pop();
101             double operand1 = operandStack.top();
```

```
102         operandStack.pop();
103
104         switch (token[0]) {
105             case '+': operandStack.push(operand1 + operand2); break;
106             case '-': operandStack.push(operand1 - operand2); break;
107             case '*': operandStack.push(operand1 * operand2); break;
108             case '/': if (operand2 == 0) return NAN; ...
                       operandStack.push(operand1 / operand2); break;
109             case '%': if (operand2 == 0) return NAN; ...
                       operandStack.push(fmod(static_cast<int>(operand1), ...
                       static_cast<int>(operand2))); break;
110             case '^': operandStack.push(pow(operand1, operand2)); ...
                       break;
111         }
112     }
113 }
114
115 if (operandStack.size() != 1) {
116     return NAN; // 操作数和操作符不匹配
117 }
118
119 return operandStack.top();
120 }
121
122 int main() {
123     string infix;
124     while (getline(cin, infix)) {
125         string postfix = infixToPostfix(infix);
126         if (postfix.empty()) {
127             cout << "ERROR IN INFIX NOTATION" << endl;
128         }
129         else {
130             double result = evaluatePostfix(postfix);
131             if (isnan(result)) {
132                 cout << "ERROR IN INFIX NOTATION" << endl;
133             }
134             else {
135                 cout << fixed << setprecision(2) << result << endl;
```

```
136         }
137     }
138 }
139
140 return 0;
141 }
```

A.2 路径简化程序

```
1 #include <iostream>
2 #include <sstream>
3 #include <vector>
4 #include <string>
5 using namespace std;
6
7 string simplifyPath(const string& path) {
8     vector<string> stack;
9     istringstream iss(path);
10    string token;
11
12    while (getline(iss, token, '/')) {
13        if (token.empty() || token == ".") {
14            continue;
15        } else if (token == "..") {
16            if (!stack.empty()) {
17                stack.pop_back();
18            }
19        } else {
20            stack.push_back(token);
21        }
22    }
23
24    if (stack.empty()) {
25        return "/";
26    }
27 }
```

```
28     string result = "";
29     for (const auto& dir : stack) {
30         result += "/" + dir;
31     }
32     return result;
33 }
34
35 int main() {
36     string path;
37     getline(cin, path);
38     string simplified = simplifyPath(path);
39     cout << simplified << endl;
40     return 0;
41 }
```